

Peretto

marcos  
Luciano

PDI — ATIVIDADE 04 · REGISTRO DE ENTREGA

# Engenharia de **Custos de LLM** (custo por tarefa, cache, roteamento)

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company — Automação & Infraestrutura

Módulo: 04 · Engenharia de IA, RAG e Vetores

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

token / cache / finanças

## 1. Contexto

LLM cobra por **token** (entrada + saída). Em produção, o custo explode silenciosamente: prompts repetidos, chamadas redundantes e modelo caro onde um barato bastaria. Reduzir custo não é "usar menos IA" — é **usar a infraestrutura certa**: cache, batch e model routing.

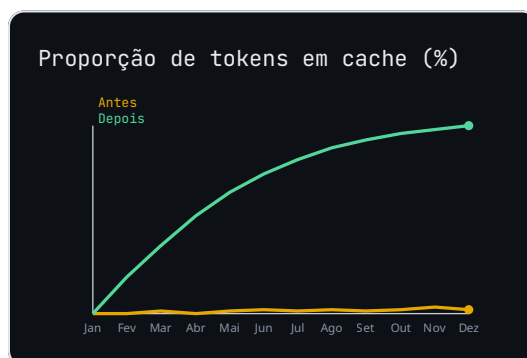
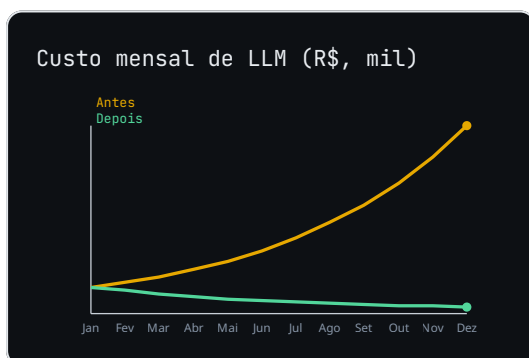
### ANALOGIA DO BUFFET

Token é a comida no prato — você paga por grama. **Cache semântico** é guardar a sobra na geladeira: se outro pede a mesma coisa, você serve do pote, não cozinha de novo. **Batch** é fazer 100 pratos de uma vez no forno, mais barato que um a um. **Model**

**routing** é mandar o prato simples pro cozinheiro júnior e o complexo pro chef — paga o chef só quando precisa.

## 2. Diagnóstico

Toda pergunta ia direto pro modelo flagship, sem reaproveitar respostas idênticas e sem agrupar chamadas. A fatura crescia 20% ao mês e ninguém sabia qual fluxo consumia mais. O custo era **opaco e não-atribuído**.



**Raiz do problema:** 100% das chamadas no modelo caro, sem cache de respostas repetidas e sem roteamento por complexidade. Custo é arquitetura, não acidente.

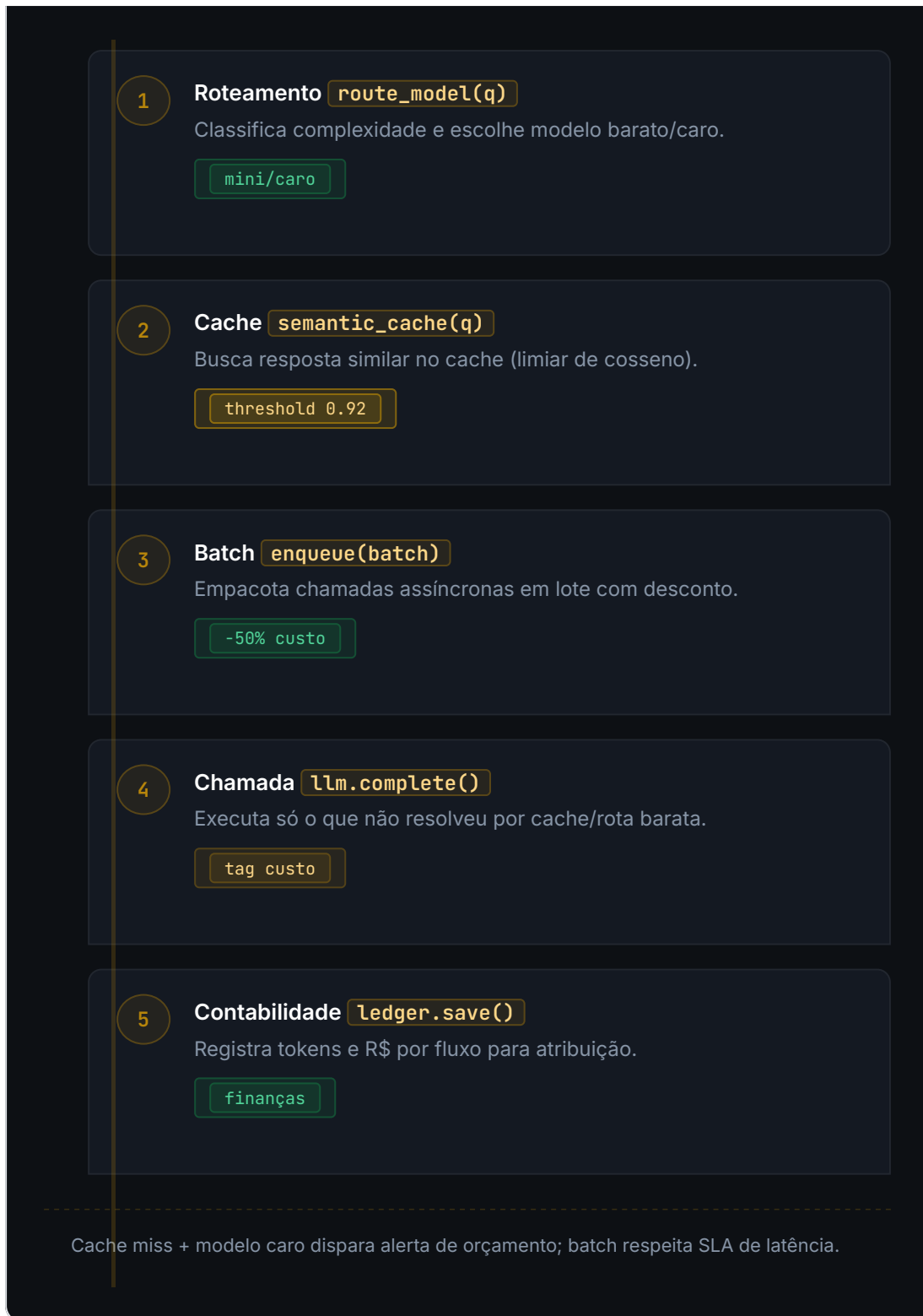
## 3. Solução

Trinca de eficiência: (1) **cache semântico** — responde do cache quando a pergunta é similar a uma já respondida; (2) **batch** — empacota chamadas não-urgentes em lotes com desconto; (3) **model routing** — classifica a pergunta e manda pro modelo barato (ex.: mini) ou caro (frontier) conforme a complexidade.

## 4. Como funciona (pipeline)

Otimização de custo de LLM

FLUXO EM EXECUÇÃO



## 5. Antes vs Depois

| CENÁRIO | ANTES (FLAGSHIP PURO) | DEPOIS (ROUTING+CACHE+BATCH) |
|---------|-----------------------|------------------------------|
|---------|-----------------------|------------------------------|

|                   |             |                    |
|-------------------|-------------|--------------------|
| Resposta repetida | Recalculada | Servida do cache   |
| Chamada simples   | Modelo caro | Modelo barato      |
| Lote              | 1 a 1       | Batch com desconto |
| Visibilidade      | Nenhuma     | Custo por fluxo    |

## 6. Entregas

- **Módulo** `model_router.py`: classificação de complexidade e roteamento.
- **Cache** `semantic_cache.py`: cache de respostas por similaridade.
- **Script** `batch_runner.py`: empacotamento de chamadas assíncronas.
- **Dashboard** `custo.md`: led de tokens e R\$ por fluxo.

## 7. Métricas



**Meta:** reduzir o custo mensal em  $\geq 85\%$  mantendo qualidade (score  $\geq 0.90$ ) via cache + routing + batch.

## 8. Status final

**NÃO PUBLICADO** — Desenvolvido e em homologação. Aguarda revisão antes de produção.